🎯 Project: Employee Management System

File Structure:

- **`EmployeeManagementSystem`** — Manages employee records and provides functionalities such as adding, searching, updating, deleting, saving, and loading employees.
- **`Employee`** — Represents an individual employee and stores details such as ID, name, department, position, salary, and joining date.

`EmployeeManagementSystem`

- **`addEmployee(Employee emp)`** — Adds a new employee to the `ArrayList` and `HashMap`.
- **`showAll()`** — Displays the details of all employees stored in the system.
- **`searchbyId(String id)`** — Searches for an employee using their ID through the `HashMap`.
- **`searchByName(String name)`** — Searches for an employee by name using the employee list.
- **`searchByDept(String department)`** — Finds all employees belonging to a specified department.
- **`updateEmployee(...)`** — Updates the details of an existing employee identified by their ID.
- **`deleteEmployee(String id)`** — Removes an employee from both the `ArrayList` and `HashMap` using their ID.
- **`saveEmployeesToFile()`** — Serializes and saves the employee list to `employees.dat`.
- **`loadEmployeesFromFile()`** — Reads the serialized employee list from `employees.dat` and rebuilds the `HashMap`.
- **`main(String[] args)`** — Provides the menu-driven interface and handles user input to perform different employee-management operations.

Code:

```
package project4;
import java.io.Serializable;
//Employee class
```

```
public class Employee implements Serializable{
	private static final long serialVersionUID = 1L;
	private String id;
  private String name;
  private String department;
  private String position;
  private double salary;
  private String joinDate;

  //Parameterized Constructors
	public Employee(String id, String name, String department, String position, double salary, String joinDate) {
		this.id = id;
		this.name = name;
		this.department = department;
		this.position = position;
		this.salary = salary;
		this.joinDate = joinDate;
	}

	//Getters and setters
	public String getId() {
		return id;
	}
	public void setId(String id) {
		this.id = id;
	}
	public String getName() {
		return name;
	}
	public void setName(String name) {
		this.name = name;
	}
	public String getDepartment() {
		return department;
	}
	public void setDepartment(String department) {
		this.department = department;
	}
	public String getPosition() {
		return position;
	}
	public void setPosition(String position) {
		this.position = position;
	}
	public double getSalary() {
		return salary;
	}
	public void setSalary(double salary) {
		this.salary = salary;
	}
	public String getJoinDate() {
		return joinDate;
	}
	public void setJoinDate(String joinDate) {
```

```
                this.joinDate = joinDate;
        }

        //Show Employee Details
        public void showDetails() {
                System.out.println("Employee id: "+id);
                System.out.println("Employee name: "+name);
                System.out.println("Department: "+department);
                System.out.println("Employee Position: "+position);
                System.out.println("Salary: "+salary);
                System.out.println("Joining Date: "+joinDate);
        }

}
```

```
package project4;
import java.util.*;
import java.io.*;

//Main class to manage Employee data
public class EmployeeManagementSystem {

        //Contain the list of employees
    List<Employee> employees=new ArrayList<>();

    //hashMap with Employee Id as key to faciliate searching of Employees using Id
    HashMap<String,Employee> map=new HashMap<>();
    String DATA_FILE = "employees.dat";

    //Method to add an Employee
    public void addEmployee(Employee emp) {
            employees.add(emp);
            map.put(emp.getId(), emp);
    }

    //Method to show all Employee Details
    public void showAll() {
            for(Employee emp:employees) {
                    emp.showDetails();
                    System.out.println("-".repeat(100));
            }
    }

    //Search Employee by Id
    public Employee searchbyId(String id) {
            return map.get(id);
   }

    //Search Employee by Name
   public List<Employee> searchByName(String name) {
            List<Employee> results=new ArrayList<>();
            for(Employee emp:employees) {
                    if(emp.getName().contains(name)) {
                            results.add(emp);
```

```
                }
        }
        return results;
  }

  //Search Employee by department
  public List<Employee> searchByDept(String department){
        List<Employee> results=new ArrayList<>();
        for(Employee emp:employees) {
                if(emp.getDepartment().equalsIgnoreCase(department)) {
                        results.add(emp);
                }
        }
   return results;
  }

  //Method to update Employee Details
  public boolean updateEmployee(String id,String name,String department,String position,double salary, String date)
{
        Employee emp=map.get(id);
        if(emp==null) {
                return false;
        }

                emp.setName(name);
                emp.setDepartment(department);
                emp.setPosition(position);
                emp.setSalary(salary);
                emp.setJoinDate(date);
                return true;
  }

  //Method to delete an employee record
  public boolean deleteEmployee(String id) {
        Employee emp=map.get(id);
        if(emp==null) {
                return false;
        }
        employees.remove(emp);
        map.remove(id);
        return true;
  }

  //Method to find number of Employees
  public int countEmployees() {
        return employees.size();
  }

  //Method to find average salary of Employees
  public double findavgsalary() {
        double total=0;
        for(Employee emp:employees) {
                total+=emp.getSalary();
        }
```

```
        double avg=total/countEmployees();
        return avg;
    }

    //Method to find Employee with highest salary
    public Employee highest() {
        double max=0;
        Employee ans=null;
        for(Employee emp:employees) {
            if(emp.getSalary()>max) {
                max=emp.getSalary();
                ans=emp;
            }
        }
        return ans;
    }

  //Method to find Employee with lowest salary
    public Employee lowest() {
        double min=10000000;
        Employee ans=null;
        for(Employee emp:employees) {
            if(emp.getSalary()<min) {
                min=emp.getSalary();
                ans=emp;
            }
        }
        return ans;
    }

    //Method to save employee details to a file
    public void saveEmployeesToFile() {
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(DATA_FILE))) {
            oos.writeObject(employees);
            System.out.println("Employee data saved to file.");
        } catch(IOException e) {
            System.out.println("Error saving employee data: " + e.getMessage());
        }
    }

    //Method to load employee details from a file
    @SuppressWarnings("unchecked")
    public void loadEmployeesFromFile() {
        File file = new File(DATA_FILE);
        if(!file.exists()) {
            System.out.println("No existing employee data found. Starting fresh.");
            return;
        }

        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(DATA_FILE))) {
            employees = (ArrayList<Employee>) ois.readObject();
            // Rebuild HashMap
            for(Employee emp : employees) {
                map.put(emp.getId(), emp);
```

```
        }
        System.out.println("Loaded " + employees.size() + " employees from file.");
    } catch(IOException | ClassNotFoundException e) {
        System.out.println("Error loading employee data: " + e.getMessage());
    }
}

//Main function to provide a menu driven interface to user
public static void main(String args[]) {
        EmployeeManagementSystem ems=new EmployeeManagementSystem();
        Scanner sc=new Scanner(System.in);
        int choice=0;
        while(choice!=9) {

                System.out.println("=== EMPLOYEE MANAGEMENT SYSTEM ===\r\n"
                                + "1. Add New Employee\r\n"
                                + "2. View All Employees\r\n"
                                + "3. Search Employee\r\n"
                                + "4. Update Employee\r\n"
                                + "5. Delete Employee\r\n"
                                + "6. Generate Reports\r\n"
                                + "7. Save to File\r\n"
                                + "8. Load from File\r\n"
                                + "9. Exit\r\n"
                                + "Enter your choice:");
                choice=sc.nextInt();

                //Implements adding an employee
                if(choice==1) {
                        System.out.println("\n===ADD EMPLOYEE===\n");
                        System.out.println("Enter Employee Id:");
                        sc.nextLine();
                        String id=sc.nextLine();
                        System.out.println("Enter Employee Name:");
                        String name=sc.nextLine();
                        System.out.println("Enter Employee Department:");
                        String dept=sc.nextLine();
                        System.out.println("Enter Employee Position:");
                        String pos=sc.nextLine();
                        System.out.println("Enter Salary");
                        double salary=sc.nextDouble();
                        System.out.println("Enter Date of Joining");
                        sc.nextLine();
                        String date=sc.nextLine();
                        Employee e1=new Employee(id,name,dept,pos,salary,date);
                        ems.addEmployee(e1);
                }

                //Implements showing employee details
                else if(choice==2) {
                        System.out.println("\n===EMPLOYEE DETAILS===\n");
                System.out.println("-".repeat(100));
                ems.showAll();
                }
```

```
//Implements Employee search
else if(choice==3) {
	int search=0;
	System.out.println("=== SEARCH EMPLOYEE ===\r\n"
			+ "1. Search by ID\r\n"
			+ "2. Search by Name\r\n"
			+ "3. Search by Department");
	search=sc.nextInt();

	//Implements Employee search by id
	if(search==1) {
		System.out.println("Enter Employee Id to search: ");
		sc.nextLine();
		String id =sc.nextLine();
	   Employee emp=ems.searchbyId(id);
		if(emp==null) {
			System.out.println("No employee found with given id!!!");
		}
		else {
			System.out.println("Employee found!!!");
			emp.showDetails();
		}
	}

	//Implements Employee search by name
	else if(search==2) {
		System.out.println("Enter Employee Name to search: ");
		sc.nextLine();
		String name =sc.nextLine();
		List<Employee> emps=ems.searchByName(name);
		if(emps==null) {
			System.out.println("No employee found with given name!!!");
		}
		else {
			System.out.println("List of employees with matching names: ");
			for(Employee emp:emps) {
				emp.showDetails();
				System.out.println("-".repeat(100));
		}
	}
	}

	//Implements Employee search by department
	else {
		System.out.println("Enter Department to search: ");
		sc.nextLine();
		String dept =sc.nextLine();
		List<Employee> results=ems.searchByDept(dept);
		if(results==null) {
			System.out.println("No employee found with Given Department!!!");
		}
		else {
			System.out.println("List of employees working in department: "+dept);
```

```
                                        for(Employee emp:results) {
                                                emp.showDetails();
                                                System.out.println("-".repeat(100));
                                        }
                                }
                        }
                }
        }

        //Implementing employee update
        else if(choice==4) {
                System.out.println("\n===UPDATE EMPLOYEE===\n");
                System.out.println("Enter id of employee to update");
                sc.nextLine();
                String id=sc.nextLine();
                System.out.println("Enter updated details");
                System.out.println("Enter Employee Name:");
                String name=sc.nextLine();
                System.out.println("Enter Employee Department:");
                String dept=sc.nextLine();
                System.out.println("Enter Employee Position:");
                String pos=sc.nextLine();
                System.out.println("Enter Salary");
                double salary=sc.nextDouble();
                System.out.println("Enter Date of Joining");
                sc.nextLine();
                String date=sc.nextLine();
                boolean found=ems.updateEmployee(id, name, dept, pos, salary, date);
                if(!found) {
                        System.out.println("No user exists with given id");
                }
                else {
                        System.out.println("Details of user with id: "+id+ " updated successfully!");
                }

        }

        //Implements deletion of employee
        else if(choice==5) {
                System.out.println("\n===DELETE EMPLOYEE===\n");
                System.out.println("Enter id of employee to delete:");
                sc.nextLine();
                String id=sc.nextLine();
                boolean found=ems.deleteEmployee(id);
                if(!found) {
                        System.out.println("No user exists with given id");
                }
                else {
                        System.out.println("User deleted successfully!");
                }
        }

        //Generate reports
        else if(choice==6) {
                System.out.println("\n===Employee Reports===\n");
```

```
                        System.out.println("Total Employees: "+ems.countEmployees());
                        System.out.println("Average Salary:  "+ems.findavgsalary());
                        Employee highest=ems.highest();
                        System.out.println("Highest Salary:  "+highest.getSalary()+ "("+highest.getName()+")");
                        Employee lowest=ems.lowest();
                        System.out.println("Lowest Salary:   "+lowest.getSalary()+ "("+lowest.getName()+")");
                }


                else if(choice==7) {
                        System.out.println("\n===SAVE EMPLOYEE DETAILS===\n");
                        ems.saveEmployeesToFile();
                }


                else if(choice==8) {
                        System.out.println("\n===LOAD EMPLOYEE DETAILS===\n");
                        ems.loadEmployeesFromFile();
                }
        }
  }


}
```

Output:

```
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
1

===ADD EMPLOYEE===

Enter Employee Id:
101
Enter Employee Name:
Alice
Enter Employee Department:
IT
Enter Employee Position:
Fresher
Enter Salary
35000
```

Enter Date of Joining
12-06-2026
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
1

===ADD EMPLOYEE===

Enter Employee Id:
102
Enter Employee Name:
Bob
Enter Employee Department:
Marketing
Enter Employee Position:
Manager
Enter Salary
75000
Enter Date of Joining
12-03-2023
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
1

===ADD EMPLOYEE===

Enter Employee Id:
103
Enter Employee Name:
Alexa
Enter Employee Department:
HR
Enter Employee Position:
Lead
Enter Salary
78000

Enter Date of Joining
12-05-2024
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
2

===EMPLOYEE DETAILS===

Employee id: 101
Employee name: Alice
Department: IT
Employee Position: Fresher
Salary: 35000.0
Joining Date: 12-06-2026

Employee id: 102
Employee name: Bob
Department: Marketing
Employee Position: Manager
Salary: 75000.0
Joining Date: 12-03-2023

Employee id: 103
Employee name: Alexa
Department: HR
Employee Position: Lead
Salary: 78000.0
Joining Date: 12-05-2024

=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
3
=== SEARCH EMPLOYEE ===
1. Search by ID
2. Search by Name

3. Search by Department
1
Enter Employee Id to search:
101
Employee found!!!
Employee id: 101
Employee name: Alice
Department: IT
Employee Position: Fresher
Salary: 35000.0
Joining Date: 12-06-2026
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
3
=== SEARCH EMPLOYEE ===
1. Search by ID
2. Search by Name
3. Search by Department
1
Enter Employee Id to search:
111
No employee found with given id!!!
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
3
=== SEARCH EMPLOYEE ===
1. Search by ID
2. Search by Name
3. Search by Department
2
Enter Employee Name to search:
Alex
List of employees with matching names:
Employee id: 103
Employee name: Alexa
Department: HR

Employee Position: Lead
Salary: 78000.0
Joining Date: 12-05-2024

=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
2

===EMPLOYEE DETAILS===

Employee id: 101
Employee name: Alice
Department: IT
Employee Position: Fresher
Salary: 35000.0
Joining Date: 12-06-2026

Employee id: 102
Employee name: Bob
Department: Marketing
Employee Position: Manager
Salary: 75000.0
Joining Date: 12-03-2023

Employee id: 103
Employee name: Alexa
Department: HR
Employee Position: Lead
Salary: 78000.0
Joining Date: 12-05-2024

=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
3
=== SEARCH EMPLOYEE ===

1. Search by ID
2. Search by Name
3. Search by Department
3
Enter Department to search:
IT
List of employees working in department: IT
Employee id: 101
Employee name: Alice
Department: IT
Employee Position: Fresher
Salary: 35000.0
Joining Date: 12-06-2026

=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
4

===UPDATE EMPLOYEE===

Enter id of employee to update
101
Enter updated details
Enter Employee Name:
Alice
Enter Employee Department:
IT
Enter Employee Position:
Fresher
Enter Salary
35000
Enter Date of Joining
12-06-2026
Details of user with id: 101 updated successfully!
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:

6

===Employee Reports===

Total Employees: 3
Average Salary: 62666.666666666664
Highest Salary: 78000.0(Alexa)
Lowest Salary: 35000.0(Alice)
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
5

===DELETE EMPLOYEE===

Enter id of employee to delete:
101
User deleted successfully!
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
7

===SAVE EMPLOYEE DETAILS===

Employee data saved to file.
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
5

```
===DELETE EMPLOYEE===

Enter id of employee to delete:
102
User deleted successfully!
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
5

===DELETE EMPLOYEE===

Enter id of employee to delete:
103
User deleted successfully!
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
2

===EMPLOYEE DETAILS===

-----------------------------------------------------------------------------------------------------
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
8

===LOAD EMPLOYEE DETAILS===
```

Loaded 2 employees from file.
=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
2

===EMPLOYEE DETAILS===

Employee id: 102
Employee name: Bob
Department: Marketing
Employee Position: Manager
Salary: 75000.0
Joining Date: 12-03-2023

Employee id: 103
Employee name: Alexa
Department: HR
Employee Position: Lead
Salary: 78000.0
Joining Date: 12-05-2024

=== EMPLOYEE MANAGEMENT SYSTEM ===
1. Add New Employee
2. View All Employees
3. Search Employee
4. Update Employee
5. Delete Employee
6. Generate Reports
7. Save to File
8. Load from File
9. Exit
Enter your choice:
9